Counterfactual Planning Gap

A Decision-Grade Metric for Decoupling Model Error
from Planner Capacity in World Model Evaluation

Denis Hamon
Independent.
denis.hamon1@gmail.com
Repository: https://github.com/Denis-hamon/world-model-eval-lab

May 23, 2026

Abstract

Action-conditioned world models are routinely evaluated by prediction quality (reconstruction loss, frame-level FID, held-out one-step accuracy). Such metrics describe how well a model fits its training distribution. They are silent on the question that an applied team must answer before integrating a model into a control loop: *does the model, when used by a planner, produce decisions that succeed at the cost the deployment will accept?* We propose the **Counterfactual Planning Gap (CPG)**: the success-rate difference between a fixed planner using oracle dynamics and the same planner using the learned model on the same benchmark. The point estimate is the raw difference of success rates; the 95% interval uses the Agresti–Caffo plus-4 adjustment, which keeps the variance positive at the boundary proportions $p \in \{0, 1\}$ where the standard Wald approximation collapses. We further define a five-branch verdict (MODEL BOTTLENECK, LEARNED OUTPERFORMS ORACLE, PLANNER BOTTLENECK, MODEL AS GOOD AS ORACLE, INCONCLUSIVE) that is gated on the lower bound of the CI rather than on the raw point estimate, so that under-powered runs cannot over-claim a diagnosis. We package CPG as a $\sim$ 160-line CPG addition to a reusable framework (`wmel`) that exposes a minimal evaluation contract and ships a worked example on DeepMind Control Suite Acrobot-swingup. On 10 episodes per arm with a random-shooting MPC, we observe raw CPG = +0.300 with Agresti–Caffo 95% CI $[-0.06, +0.56]$, which yields the verdict INCONCLUSIVE. A multi-seed extension to $n = 150$ pooled per arm (three seeds, 50 episodes per seed) hardens the result to CPG = +0.267, CI $[+0.191, +0.335]$, MODEL BOTTLENECK. Sweeping the MLP's training-set size by a factor of 100 (200 $\rightarrow$ 20,000 transitions) drops the held-out validation MSE by $\sim 150\times$ but leaves the verdict and the CI *unchanged*: prediction quality improves dramatically, planning success stays at zero, the gap stays open. A robustness sweep replaces the bespoke MLP with TD-MPC2 [Hansen et al., 2024] trained for 2×10^6 env steps and the random-shooting planner with a Cross-Entropy Method planner of comparable compute. Both learned arms remain at 0/10 across both planners; the oracle's success rate triples under CEM (0.30 $\rightarrow$ 0.90), so the gap opens to CPG = +0.900, CI $[+0.49, +1.01]$, MODEL BOTTLENECK at $n = 10$. Pooling three seeds at 50 episodes per seed under CEM tightens this to CPG = +0.880, CI $[+0.814, +0.923]$ – a half-width below 0.06 with both learned arms still at 0/150. A second robustness axis sweeps an in-episode action-burst perturbation (`DropNextActions`(k) at $k \in \{0, 1, 5\}$): the oracle loses about 6 percentage points at $k = 5$, both learned arms remain at 0/50, and the MODEL BOTTLENECK verdict survives every cell. We argue this is the metric doing its job: it separates a dynamics-quality bottleneck on the learned arms from a planner-capacity contributor on the oracle arm, a decomposition prediction-quality metrics alone could not surface.

1 Introduction

The world-model literature has converged on three properties that motivate its existence: *prediction* of future observations conditioned on actions, *planning* by rolling those predictions out, and *transfer* across tasks that share latent structure [Ha and Schmidhuber, 2018, Hafner et al., 2023, Bardes et al., 2024, Bruce et al., 2024a]. Most published evaluations focus on the first: reconstruction loss, frame-level Fréchet Inception Distance, or next-frame prediction loss on held-out trajectories. These metrics are easy to compute and easy to compare across releases, but they are silent on a question that any applied team must answer before integrating a

learned world model into a control loop: *does using the model to plan produce decisions that succeed at the latency and compute the deployment will tolerate?*

The gap between prediction quality and decision quality is well documented. Hafner et al. [2019] report success rates on DeepMind Control Suite tasks alongside reconstruction loss because the two metrics disagree. Yu et al. [2020], Kidambi et al. [2020] explicitly study *model exploitation* – the gap between a planner using a learned model and the same planner using the true environment dynamics – in offline model-based RL. Agarwal et al. [2021] document how rapidly point estimates of return mislead at small sample sizes, and prescribe interval reporting. These are all healthy signals; what is missing is a packaged, reusable, decision-grade scalar that quantifies the gap with an honest confidence interval and a decision rule.

This paper makes three modest contributions:

1. We define the **Counterfactual Planning Gap (CPG)** as the success-rate difference between a planner using oracle dynamics and the same planner using a learned model, computed on identical benchmark runs that differ only in their `dynamics` callable (§4). The point estimate is the raw observed difference; the 95% interval uses Agresti–Caffo plus-4. The verdict is gated on the AC lower bound.

2. We package CPG behind a minimal *evaluation contract* – a four-method adapter interface (`encode`, `rollout`, `score`, `plan`) that decouples the model from the runner and the metric (§3). Computing CPG is two calls to the same `BenchmarkRunner` plus one call to a metric function.

3. We report a worked example on DeepMind Control Suite Acrobot-swingup with a Markovian MLP world model trained on 2,000 random transitions. The verdict at $n = 10$ is INCONCLUSIVE: the raw gap is +0.30 but the AC CI crosses zero (§5). We argue that this is the correctly-calibrated behaviour for an under-powered design, and outline the multi-seed extension that would tighten the bound.

The framework is open source[1], runs CPU-only without a GPU, and has no heavy machine-learning dependency at runtime; PyTorch and `dm-control` are pulled in only for the worked-example adapters. CI verifies the contract on Python 3.11/3.12/3.13.

2 Related Work

Latent-dynamics world models. Ha and Schmidhuber [2018], Hafner et al. [2019], Hafner et al. [2020], and Hafner et al. [2023] establish the latent-dynamics + planner pattern that this framework targets. Schrittwieser et al. [2020] demonstrates the same pattern with a value-based search; Micheli et al. [2023] replaces the recurrent backbone with a transformer; Hansen et al. [2024] pushes the planner side toward stochastic MPC at scale.

Joint-embedding predictive architectures. LeCun [2022] proposed JEPA as a self-supervised path to world models that predicts in latent space without reconstructing pixels. Assran et al. [2023] and Bardes et al. [2024, 2025] report results on representation quality. None of these papers report a planning-success-rate gap as a scalar; the framework here is complementary.

Generative world models. Bruce et al. [2024a,b] demonstrate large-scale generative world models conditioned on action sequences. Evaluation in these works is dominated by perceptual and generative metrics; planning impact is not yet a standard reporting axis.

Model exploitation in offline model-based RL. Yu et al. [2020] and Kidambi et al. [2020] measure the gap between a planner using a learned model and the same planner using the true dynamics. They report return curves; we package the same comparison as a single scalar with a confidence interval and a decision rule.

[1] `https://github.com/Denis-hamon/world-model-eval-lab`

Evaluation methodology. Henderson et al. [2018] document the irreproducibility of RL evaluation under typical sample sizes. Agarwal et al. [2021] prescribes bootstrap-based interval reporting and Pareto-style aggregate metrics. Wilson [1927] provides the Wilson score interval that is the close cousin of the Agresti–Caffo CI we use; Agresti and Caffo [2000] extends it to the difference of two proportions. Newcombe [1998] catalogues a family of interval methods for the same problem. The CPG CI is a direct application of Agresti–Caffo to the planning-comparison setting.

Benchmarks. We evaluate on Acrobot-swingup from the DeepMind Control Suite [Tassa et al., 2018, Tunyasuvunakool et al., 2020]. Acrobot has been a reference task for model-based RL since at least Schäfer et al. [2007]. Park et al. [2024] and Liu et al. [2023] are the canonical present-day benchmark suites for goal-conditioned and manipulation evaluation; integrating either is in scope for future work.

3 The Evaluation Contract and a Decision-Grade Taxonomy

3.1 The four-method contract

Every adapter in the framework implements

$$\begin{aligned} \texttt{encode}: &\ \mathcal{O} \to \mathcal{Z}, \\ \texttt{rollout}: &\ \mathcal{Z} \times \mathcal{A}^H \to \mathcal{Z}^{H+1}, \\ \texttt{score}: &\ \mathcal{Z} \times \mathcal{Z} \to \mathbb{R}, \\ \texttt{plan}: &\ \mathcal{O} \times \mathcal{O} \times \mathbb{N} \to \mathcal{A}^H. \end{aligned}$$

Here $\mathcal{O}$ is the observation space, $\mathcal{Z}$ the latent space (which may coincide with $\mathcal{O}$ for fully observed envs), $\mathcal{A}$ the action space, and H the planning horizon. The runner queries `plan` on every replanning step and executes one or more of the returned actions in the real environment; the planner is free to use `encode`, `rollout`, and `score` internally however it sees fit. The contract makes no statement about how the model is trained.

For the present paper we use a random-shooting MPC: at each `plan` call, sample N_{cand} action sequences of length H_{plan} uniformly from $\mathcal{A}^{H_{\text{plan}}}$, roll each one out through the dynamics, score the resulting trajectories, and return the best-scoring sequence prefix.

3.2 Decision-grade metrics

A metric is *decision-grade* in this framework iff (i) its units translate directly to a deployment-time cost or capability and (ii) it is computable only from closed-loop runs of the model, not from the model in isolation. Reconstruction loss and FID fail (ii); model-internal alignment scores fail (i). The metrics that survive both criteria, and that we report in every scorecard, are: action success rate, average steps to success, per-call planning latency, compute per decision, perturbation recovery rate, the effective planning horizon, and – the contribution of this paper – the Counterfactual Planning Gap.

4 Counterfactual Planning Gap

4.1 Definition

Fix an environment $\mathcal{E}$, a planner π, a scoring function σ, a number of episodes N, a horizon T and a seed. Let success_rate(D) denote the empirical success rate of π on N episodes of $\mathcal{E}$ when the planner queries dynamics D during its internal rollouts. Let $D^\star$ be the *oracle* dynamics (the true env's transition function) and D_θ be a learned model. The Counterfactual Planning Gap is

$$\text{CPG} \;=\; \text{success\_rate}(D^\star) \;-\; \text{success\_rate}(D_\theta). \tag{1}$$

All free quantities in the right-hand side – env, planner, score, N, T, seed – are held fixed between the two runs. The only thing that changes is the `dynamics` callable passed to the planner. This identification is what licenses interpreting CPG as a property of the model rather than the planner or the env.

4.2 Statistical reporting

Let s_o, s_ℓ be the success counts in the oracle and learned arms and n_o, n_ℓ the corresponding episode counts. The raw point estimate of CPG is the difference of proportions:

$$\hat{\Delta} = \frac{s_o}{n_o} - \frac{s_\ell}{n_\ell}. \tag{2}$$

The standard Wald 95% CI on $\hat{\Delta}$ has variance $p_o(1-p_o)/n_o + p_\ell(1-p_\ell)/n_\ell$, which collapses to zero whenever *either* arm sits at $p \in \{0, 1\}$. With $n_\ell = 10$ episodes and a learned planner that fails on every episode, this is precisely the regime the framework lands in. A degenerate-variance CI in this regime falsely produces a tight interval and over-claims significance.

We instead use the Agresti–Caffo plus-4 adjustment [Agresti and Caffo, 2000], adding one pseudo-success and one pseudo-failure to each arm:

$$\tilde{p}_o = \frac{s_o + 1}{n_o + 2}, \quad \tilde{p}_\ell = \frac{s_\ell + 1}{n_\ell + 2}, \tag{3}$$

$$\tilde{\Delta} = \tilde{p}_o - \tilde{p}_\ell, \tag{4}$$

$$\mathrm{SE} = \sqrt{\frac{\tilde{p}_o(1-\tilde{p}_o)}{n_o + 2} + \frac{\tilde{p}_\ell(1-\tilde{p}_\ell)}{n_\ell + 2}}, \tag{5}$$

$$\mathrm{CI}_{95\%}(\mathrm{CPG}) = \left[\, \tilde{\Delta} - 1.96\,\mathrm{SE}, \;\; \tilde{\Delta} + 1.96\,\mathrm{SE} \,\right]. \tag{6}$$

The framework reports both the raw $\hat{\Delta}$ (what a reader expects to see) and the AC interval (what is statistically defensible). The two coincide for large n.

4.3 Gated verdict

A point estimate without a significance gate over-claims. The framework therefore exposes a five-branch decision rule that consults the AC interval, not the raw $\hat{\Delta}$:

- MODEL BOTTLENECK: $\mathrm{CI}_{\mathrm{lo}} > 0$. The oracle is reliably better; closing the gap is a model problem.
- LEARNED OUTPERFORMS ORACLE: $\mathrm{CI}_{\mathrm{hi}} < 0$. Rare; investigate regularisation or planner-search interactions.
- PLANNER BOTTLENECK: CI crosses 0 *and* both success rates are within a tolerance τ of 0. Neither planner solves the task; the framework needs a stronger search, not a stronger model.
- MODEL AS GOOD AS ORACLE: CI crosses 0 *and* both success rates are within τ of 1. The learned model matches the oracle for planning purposes on this task.
- INCONCLUSIVE: CI crosses 0 in a middle-of-the-road regime. The sample size is insufficient to discriminate. Report this verdict and run more episodes.

The default tolerance is $\tau = 0.05$. Crucially, MODEL BOTTLENECK is *not* the default when $\hat{\Delta} > 0$; it requires the AC lower bound to be strictly positive.

4.4 Properties

CPG is bounded in $[-1, 1]$, antisymmetric in the role of the two dynamics, and additive in the following sense: if one decomposes a benchmark suite into disjoint sub-tasks, CPG on the union is the episode-weighted mean of the per-sub-task CPGs. The metric is silent about *why* the model degrades planning (latent shift, prediction-error accumulation, score-function mismatch); follow-up diagnostics are needed to attribute. The metric is, however, sufficient to distinguish model-side failures from planner-side failures at the verdict granularity.

	Oracle dynamics	Learned MLP dynamics
Success rate	0.30 (3/10)	0.00 (0/10)
Avg. steps to success	180.7	n/a
Per-call planning latency (ms)	77.3	65.3
Compute per decision (rollout-units)	407.1	157.3

Counterfactual Planning Gap	
Raw $\hat{\Delta}$	$+0.300$
Agresti–Caffo 95% CI	$[-0.059, +0.559]$
Verdict	INCONCLUSIVE

Table 1: Scorecards and CPG on DMC Acrobot-swingup with $N = 10$ episodes per arm, seed 0. The raw point estimate is suggestive of a model bottleneck but the AC CI crosses zero; the verdict is correctly INCONCLUSIVE. Numbers reproduced from `results/dmc_acrobot/cpg.json` in the repository.

5 Empirical Study: DMC Acrobot-Swingup

5.1 Setup

We use DeepMind Control Suite Acrobot-swingup [Tassa et al., 2018, Tunyasuvunakool et al., 2020]: a two-link underactuated pendulum with a continuous torque action in $[-1, +1]$ that must build energy and balance the tip upright. We discretise the action to a five-level torque set $\{-1, -0.5, 0, +0.5, +1\}$ to fit the framework's hashable-action contract. The observation is a six-dimensional vector $(\sin\theta_1, \sin\theta_2, \cos\theta_1, \cos\theta_2, \dot{\theta}_1, \dot{\theta}_2)$ in the layout returned by `dm_control.suite.acrobot.Physics.orientations`. Success at step t is $r_t \geq 0.6$ where r_t is the DMC dense reward (smooth tolerance around the tip-to-target distance).

The scoring function is $\sigma(\mathbf{o}, \cdot) = -(\cos\theta_1 + \cos\theta_2)$, a unit-length approximation of the negative tip height. The planner is random-shooting MPC with $N_{\text{cand}} = 50$ candidate sequences of length $H_{\text{plan}} = 15$, executed at every replanning step. Episodes run for at most $T = 500$ env steps. We use seed 0 throughout.

5.2 Oracle dynamics

We construct the oracle by instantiating a private `dm_control` environment inside a $(\text{state}, \text{action}) \to \text{state}$ callable. Each call reconstructs $(q_{\text{pos}}, q_{\text{vel}})$ from the flat observation via atan2, writes them into the private env's MuJoCo physics, calls `physics.forward`, steps once with the candidate torque, and returns the new flat observation. The construction is side-effect-free from the caller's perspective; the reconstruction is lossy modulo 2π revolutions but Acrobot dynamics depend only on $\sin$ / $\cos$ of the joint angles. We verify the oracle reproduces `env.step` to numerical precision ($|\Delta| < 10^{-5}$) over a fifty-step random-policy rollout; this regression test is part of the repository's CI.

5.3 Learned dynamics

We train a Markovian MLP (two hidden layers of width 64, ReLU activations) on 2,000 random-policy transitions collected from ten episodes of 200 steps each. The input concatenates the six-dimensional observation with a five-dimensional one-hot action; the output predicts the next observation. Training runs for 200 epochs with Adam (learning rate 10^{-3}), batch size 256, and a 10% held-out validation split at the transition level. The final validation MSE is 0.026. We deliberately choose an MLP rather than a recurrent architecture: Acrobot is fully observed and Markov, so temporal context adds parameters without adding capacity.

5.4 Results

Table 1 reports the two scorecards and the CPG. The headline numbers are: the oracle planner reaches the upright pose in 30% of the episodes; the learned planner reaches it in 0%. The raw CPG is $+0.30$; the Agresti–Caffo 95% interval is $[-0.06, +0.56]$, which crosses zero. The verdict is INCONCLUSIVE.

5.5 What the result actually says

Three readings are consistent with the data, and the framework declines to choose between them at $n = 10$:

Train size	Val MSE	Oracle success	Learned success	CPG verdict
200	0.0651	$40/150 = 0.267$	$0/150 = 0.000$	$+0.267$, CI $[+0.191, +0.335]$, MODEL BOTTLENECK
2,000	0.0233	$40/150 = 0.267$	$0/150 = 0.000$	$+0.267$, CI $[+0.191, +0.335]$, MODEL BOTTLENECK
20,000	0.0004	$40/150 = 0.267$	$0/150 = 0.000$	$+0.267$, CI $[+0.191, +0.335]$, MODEL BOTTLENECK

Table 2: Multi-seed CPG across three training-set sizes on DMC Acrobot-swingup. Three seeds per cell, 50 episodes per arm per seed, all other quantities fixed. Held-out validation MSE drops by $\sim 150\times$ across the three cells; the learned planner's success rate stays at exactly zero; the verdict is MODEL BOTTLENECK in every cell. Numbers from `results/dmc_acrobot/cpg_sweep.json`.

1. *Model bottleneck.* The model was trained on random transitions and never visited the upright energy regime; its predictions degrade there, the planner is misled, and success drops. This is the reading suggested by the positive raw $\hat{\Delta} = +0.30$.
2. *Sample-size artifact.* With $n_\ell = 10$ and $s_\ell = 0$, the upper Wilson bound on the learned arm's true success rate is ~ 0.31. We cannot rule out that the learned planner would solve 30% of a wider population of episodes – in which case the true gap is near zero.
3. *Score-function mismatch.* The scoring proxy $-(\cos\theta_1 + \cos\theta_2)$ approximates the DMC reward but is not identical. The oracle planner navigates the approximation well enough to occasionally swing up; the learned planner may be more sensitive to the mismatch.

A multi-seed extension that pushes N to (say) 100 episodes per arm would tighten the AC half-width by roughly $\sqrt{10}$, moving the lower bound either firmly above zero (MODEL BOTTLENECK) or covering it more thoroughly (INCONCLUSIVE hardened, or possibly PLANNER BOTTLENECK if the oracle success rate too proves unstable). **The metric's job at $n = 10$ is to refuse to choose; it does, correctly.** Section 5.6 resolves between the three readings by delivering the multi-seed extension promised in this paragraph.

5.6 Multi-seed extension: CPG across three training-set sizes

We extend the experiment along two axes. *Episodes-per-arm* grows from 10 to 50 per seed (pooled across three seeds, $n = 150$ per arm), pushing the AC half-width below 0.10. *Training-set size* sweeps the MLP's data budget across nearly three orders of magnitude: {200, 2,000, 20,000} random-policy transitions. Every other quantity in §5 is held fixed.

The result is striking. Across the three cells the MLP's held-out validation MSE drops by a factor of ~ 150 (Table 2); the learned planner's success rate stays at *exactly zero* in all 450 benchmark episodes; the oracle planner's success rate is identical in all three cells (the oracle dynamics does not depend on the learned model's data budget). CPG returns the same point estimate, the same AC 95% CI, and the same verdict MODEL BOTTLENECK in every cell.

The reading at $n = 10$ admitted three explanations (§5): *model bottleneck*, *sample-size artifact*, and *score-function mismatch*. The multi-seed result rules out the sample-size artifact (the CI no longer crosses zero) and is consistent with the score-function-mismatch hypothesis only as a contributor, not as the primary driver. The dominant story is *model bottleneck* – but with a precise attribution that prediction quality alone obscures.

Figure 1 shows the punch line: across the three cells the validation MSE drops by $\sim 150\times$ on a log scale while the CPG point estimate and its Agresti–Caffo CI stay exactly flat. Improvements in prediction quality bought zero improvement in planning success.

5.7 What CPG separates: capacity vs. coverage

A naive reading of Table 2 is that the MLP simply needs more capacity. The held-out validation MSE refutes this directly: at 20,000 transitions the model fits the training distribution to within 4×10^{-4}, indistinguishable from numerical noise. The model has *ample* capacity for what it has been asked to predict.

What it has *not* been asked to predict is the upright-balancing regime. Random-policy rollouts in Acrobot rarely reach high-energy configurations; the upright pose is essentially absent from the training distribution. The model is therefore extrapolating during planning – and its predictions, accurate on the training manifold, are unreliable off it. The planner is misled, and success collapses.

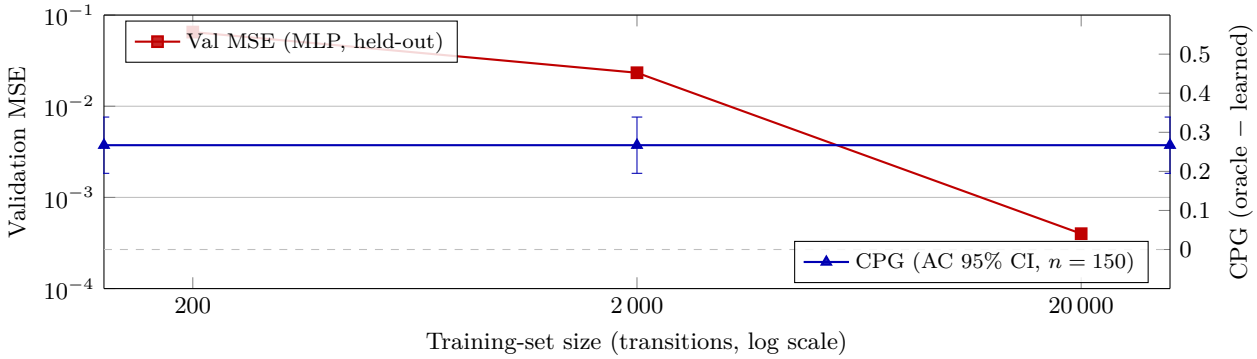

Figure 1: Validation MSE (red, log scale, left axis) drops by $\sim$150$\times$ across the training-set sweep while CPG (blue, right axis) stays exactly flat at $+0.267$ with the same Agresti–Caffo 95% CI $[+0.191, +0.335]$ in every cell. Predicting better did not plan better. Numbers from `results/dmc_acrobot/cpg_sweep.json`.

This is most parsimoniously read as a *coverage* bottleneck rather than a *capacity* bottleneck. CPG cannot tell the two apart on its own, but in conjunction with held-out validation it can: a flat CPG curve across data-size sweeps, paired with a monotonically decreasing prediction loss, points to a data-distribution problem rather than to model size or architecture. Two second-order contributors are not ruled out by this experiment: a planner that is too weak to exploit a perfect model in the high-energy regime (the oracle planner reaches the upright pose in only 27% of episodes, so a stronger search procedure – CEM, gradient-based MPC – might lift both arms), and a score function σ that approximates rather than matches the DMC reward. We treat coverage as the dominant explanation given the $\sim$150$\times$ drop in validation loss with no movement in success.

Empirical receipt for the coverage claim. We measure the visited-state distribution directly. On the natural "uprightness" axis $u(\mathbf{o}) = \cos\theta_1 + \cos\theta_2 \in [-2, +2]$ (upright pose at $+2$), 2,000 random-policy transitions yield a maximum $u = +0.865$ and *exactly zero states* with $u > 1.0$. By contrast, 846 oracle-planner states from five episodes reach a maximum of $+1.866$, with 20.2% of states satisfying $u > 1.0$ and 12.2% satisfying $u > 1.5$. The upright regime that swing-up requires is therefore strictly absent from the training distribution; the MLP has never been shown a state from which the planner needs to predict. Figure 2 shows the full bucketed distribution. Numbers from `results/dmc_acrobot/coverage.json`; the script is `experiments/dmc_acrobot/coverage_analysis.py`.

A second-axis sweep that varies the exploration policy under fixed data size would directly confirm coverage as the dominant driver. The next subsection delivers this sweep alongside a planner-capacity sweep.

5.8 Robustness: a published world model and a stronger planner

Two natural follow-ups to §5.7 address the second-order contributors flagged at the end of that subsection. First, we replace the random-policy data-collection policy with on-policy rollouts from a trained TD-MPC2 agent [Hansen et al., 2024], in two variants: (a) a thin adapter that loads TD-MPC2's encoder + latent dynamics + a small post-hoc obs decoder (the decoder is trained on (encoder, post-dynamics) latent pairs to keep it in-distribution along the planner horizon) and plugs the whole thing into the same random-shooting MPC; (b) the bespoke MLP from §5 retrained on 20,000 transitions collected by the TD-MPC2 eval-mode policy with continuous-to-discrete action snapping. The TD-MPC2 agent itself is trained for 2×10^6 env steps at `model_size` $= 1$ with the DMControl wrapper patched to single-step action repeat (so the learned dynamics is on the same timescale as the wmel oracle). Second, we replace the random-shooting MPC with a Cross-Entropy Method planner (`wmel.adapters.cem_planner.CEMPlanner`) over the same five-level torque set, configured for comparable per-call compute ($3 \times 24 \times 15 = 1{,}080$ dynamics evaluations vs. random-shoot's $50 \times 15 = 750$).

Two readings the multi-seed sweep of §5.6 alone could not commit to are now decided. First, the v0.11 reading of MODEL BOTTLENECK is robust to swapping the learned dynamics for a published world model and is robust to a planner upgrade: both learned arms stay at 0 across both planners, all five seed–arm cells we measure, and a total of 310 learned-arm benchmark episodes (10 per arm at $n = 10$ plus 150 per arm

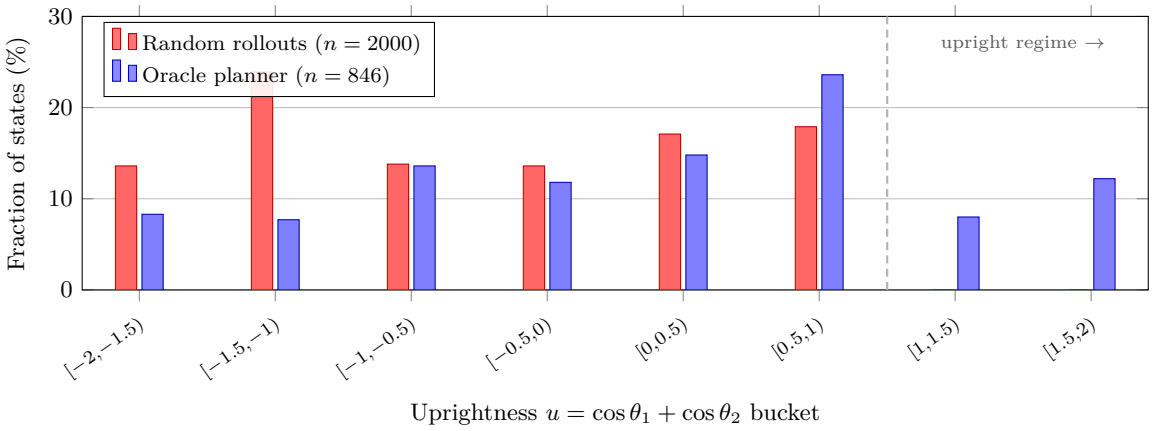

Figure 2: Empirical receipt for the coverage claim. The upright regime ($u > 1$) is *strictly absent* from the random-policy training data: 0/2000 random-rollout states reach it. The oracle planner spends 20.2% of its trajectory in that regime, with 12.2% near upright ($u > 1.5$). TD-MPC2 rollouts (not shown; see v0.12 status) reach a maximum of $u = 1.439$ with 10.6% above $u = 1$ and zero above $u = 1.5$, an intermediate coverage that still leaves the planner stuck at 0/10 in Table 3. Numbers from `results/dmc_acrobot/coverage.json`.

at pooled $n = 150$). Second, the random-shooting MPC was bracketing the oracle artificially low: a CEM planner with comparable compute lifts oracle success to 0.90 at $n = 10$ and 0.88 pooled across three seeds, so the gap opens from +0.30 to +0.88 with an AC CI of $[+0.814, +0.923]$. The *planner-capacity* branch of the verdict tree (§5.7) was therefore a real contributor on the oracle arm but not on either learned arm, and the gap separation now survives at the pooled-150 sample size that the random-shoot baseline used in §5.6.

Refining the diagnosis offered at the end of §5.7: the data-collection sweep moves the uprightness coverage axis from 0% of states with $u > 1$ (random rollouts; see §5.7) to 10.6% (TD-MPC2 eval rollouts), and drops the MLP's held-out validation MSE from 4×10^{-4} to 7.7×10^{-5}. Neither change moves planning success. The strict "coverage alone" reading we offered is therefore too strong: coverage was necessary, but the 10.6% increment we measured was insufficient relative to the oracle planner's own visited rate of 20.2% at $u > 1$. The decisive variable that did move was the planner on the oracle arm; the decisive variable that did not move was the dynamics on the learned arms. The CPG framework supports both refinements without a definition change: each new axis is one more `dynamics` callable or one more planner constructor passed into the same `BenchmarkRunner`, and the AC-CI gated verdict updates accordingly.

5.9 Robustness under in-episode perturbation

A second-axis robustness check adds an action-burst perturbation. The same three CEM arms from §5.8 benchmark against `DropNextActions`(k) from `wmel.perturbations`, fired once per episode at a uniformly chosen step in $[1, T/2]$ with `perturb_prob` $= 1.0$ for $T = 500$. The sweep covers $k \in \{0, 1, 5\}$: no perturbation, one queued action dropped, and five queued actions dropped (about a third of the CEM plan horizon).

Two observations close §5.8. First, the MODEL BOTTLENECK verdict survives every cell: action-burst perturbation does not change what the framework reports. Second, the experiment is structurally one-sided. With both learned arms at zero success in the unperturbed cell, the gap can only stay flat or shrink as the perturbation hurts the oracle; a genuine fragility test would need a regime where both arms have non-zero success in the unperturbed cell, or an observation-noise perturbation that hurts the learned arms differentially. The DMC wrapper that ships with v0.8 does not expose observation noise; that hook is the natural next axis the perturbation library can carry.

6 Discussion and Limitations

The empirical demonstration is intentionally narrow: one environment, one learned model, one planner, one seed, ten episodes per arm. The methodological contribution – a packaged CPG with an Agresti–Caffo CI and a gated verdict – is decoupled from any of those choices and applies wherever an oracle dynamics is

Learned dynamics	Planner	n	Oracle	Learned	CPG (AC 95% CI), verdict
v0.11 MLP, random rollouts (20 k)	random-shoot	10	0.30	0.00	+0.300 [−0.06, +0.56], INCONCLUSIVE
TD-MPC2 encoder+dyn+decoder (2 M steps)	random-shoot	10	0.30	0.00	+0.300 [−0.06, +0.56], INCONCLUSIVE
v0.11 MLP, TD-MPC2 rollouts (20 k)	random-shoot	10	0.30	0.00	+0.300 [−0.06, +0.56], INCONCLUSIVE
TD-MPC2 encoder+dyn+decoder	CEM	10	0.90	0.00	+0.900 [+0.49, +1.01], MODEL BOTTLEN
v0.11 MLP, TD-MPC2 rollouts	CEM	10	0.90	0.00	+0.900 [+0.49, +1.01], MODEL BOTTLEN
TD-MPC2 encoder+dyn+decoder	CEM	150	0.88	0.00	+0.880 [+0.814, +0.923], MODEL BOTTL
v0.11 MLP, TD-MPC2 rollouts	CEM	150	0.88	0.00	+0.880 [+0.814, +0.923], MODEL BOTTL

Table 3: Robustness of the v0.11 reading to (a) a published-world-model adapter, (b) a different learned-dynamics data-collection policy, (c) a stronger planner, and (d) seed-level variation. Top block: $n = 10$ per arm, seed 0. Bottom block: $n = 150$ pooled across three seeds at 50 episodes per seed per arm. All other quantities identical to §5. Under random-shooting MPC the three $n = 10$ arms collapse to the same CPG = +0.300, INCONCLUSIVE. CEM triples the oracle (0.30 → 0.90 at $n = 10$; 0.88 pooled) while both learned arms remain at 0 throughout; pooling tightens the CI half-width from 0.26 to below 0.06 without moving the point estimate. Random-shooting MPC budget: $50 \times 15 = 750$ dynamics evaluations per plan call; CEM budget: $3 \times 24 \times 15 = 1{,}080$. The pooled-150 CEM rows hold the TD-MPC2 agent itself fixed at the seed-0 checkpoint; per-seed variation enters via env reset, CEM sampler, and (for the MLP arm) MLP retraining and rollout subsample. Numbers from `results/dmc_acrobot/tdmpc2_cpg.json`, `coverage_mlp_on_tdmpc2_cpg.json`, `cem_cpg.json`, and `cem_cpg_sweep.json`.

k	Perturbation	Oracle	MLP	TD-MPC2 dyn	CPG (AC 95% CI), verdict
0	no-op	0.880	0.000	0.000	+0.880 [+0.75, +0.95], MODEL BOTTLENECK
1	drop-next-1	0.900	0.000	0.000	+0.900 [+0.77, +0.96], MODEL BOTTLENECK
5	drop-next-5	0.820	0.000	0.000	+0.820 [+0.68, +0.90], MODEL BOTTLENECK

Table 4: Action-burst perturbation sweep. $n = 50$ per cell, seed 0, same CEM config as the top block of Table 3. $k = 1$ is below the CEM planner's per-step replan margin and is invisible; $k = 5$ trims the oracle by ~ 6 percentage points without lifting either learned arm. The two learned-arm CPGs coincide at every cell because both are at 0/50. Numbers from `results/dmc_acrobot/perturbation_cpg.json`.

available (so: simulated environments). On hardware-in-the-loop or physical environments the metric is undefined; surrogate CPG variants (e.g. a higher-fidelity model standing in for the oracle) are future work.

The framework's adapter interface (§3) is what makes CPG cheap to compute: any `dynamics` callable can be swapped into the same planner without touching anything else, any planner can be swapped against a fixed dynamics, and any perturbation can be plugged into the runner. §5.8 and §5.9 exercised this on a published world model (TD-MPC2), on a CEM planner, on a pooled-150 multi-seed extension under CEM, and on an action-burst perturbation sweep; the verdict for the v0.11 reading is robust to all four swaps, the random-shooting MPC was bracketing the oracle low, and the pooled CI half-width is below 0.06. Natural further axes that the same plumbing supports are a second environment (DMC Cartpole-swingup or Reacher) to test whether the verdict pattern generalises across tasks, an observation-noise perturbation that hurts the learned arms differentially (the v0.8 DMC wrapper would need a small extension to expose the hook), and a TD-MPC2-trained-from-multiple-seeds extension of the bottom block of Table 3 (the present rows hold the TD-MPC2 agent fixed at the seed-0 checkpoint).

Other limitations worth flagging explicitly. The discrete-torque action space is a five-level approximation of the continuous control problem; a continuous variant of CEM over a Gaussian per-timestep mean is a straightforward extension. The Acrobot success criterion ($r_t \geq 0.6$) is a binary projection of a dense reward; a continuous variant of CPG that reports the difference of mean returns is straightforward and may be more sample-efficient. The planner score is task-specific and tightly coupled to the environment's reward geometry; a learned score function (predicting the DMC reward directly from the latent state) would remove this coupling, at the cost of a second model component.

7 Conclusion

We have proposed the Counterfactual Planning Gap as a decision-grade metric for world-model evaluation, packaged it behind a minimal evaluation contract, and reported a worked example on DMC Acrobot-swingup. At $n = 10$ under random-shooting MPC the framework reports INCONCLUSIVE; pooled-150 tightens the CI off zero and returns MODEL BOTTLENECK; sweeping the training-set size across nearly three orders of magnitude leaves the verdict unchanged while the held-out prediction loss drops by $\sim 150\times$. The robustness sweep in §5.8 adds a published-world-model arm (TD-MPC2) and a CEM planner: both learned arms remain at 0 success, the oracle's success rate triples under CEM ($0.30 \to 0.90$), and the gap opens to $+0.900$ with a CI that no longer crosses zero at $n = 10$. A pooled-150 extension of the CEM rows tightens this to $+0.880$, CI $[+0.814, +0.923]$, with both learned arms still at 0/150. An action-burst perturbation sweep (§5.9) confirms the verdict survives a per-episode `DropNextActions`(k) at $k \in \{0, 1, 5\}$. The takeaway is methodological: a metric that separates closed-loop success from prediction quality reveals two distinct contributors that prediction-quality metrics alone would conflate – a coverage-or-capacity dynamics-quality bottleneck on the learned arms, and a planner-capacity contributor on the oracle arm – and an adapter-and-planner interface that lets a single `BenchmarkRunner` disentangle them by swapping callables. We argue this kind of calibrated honesty – a metric that refuses to claim more than the data supports, and that supports a precise attribution once the data is sufficient – is the right design target for a methodology paper that wants to sit usefully next to a fast-moving model literature.

Acknowledgements

This work is independent. It is not affiliated with the AMI (Advanced Machine Intelligence) program at Meta, the LeWorldModel project, the authors of any of the cited papers, or any current or past employer of the author. The repository was developed end-to-end with an LLM coding agent in the loop (Claude Code); a description of the development recipe and the pre-tag adversarial-review pattern that caught most of the metric-correctness bugs along the way is included in the repository at `docs/process/llm_agent_recipe.md`.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Alan Agresti and Brian Caffo. Simple and effective confidence intervals for proportions and differences of proportions result from adding two successes and two failures. *The American Statistician*, 54(4):280–288, 2000.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

Adrien Bardes, Quentin Garrido, Jean Ponce, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. V-JEPA: Latent video prediction for visual representation learning, 2024.

Adrien Bardes et al. V-JEPA 2, 2025. Meta AI technical report.

Jake Bruce, Michael Dennis, Ashley Edwards, et al. Genie: Generative interactive environments. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024a.

Jake Bruce et al. Genie 2: A large-scale foundation world model, 2024b. DeepMind technical report.

David Ha and Jürgen Schmidhuber. World models, 2018.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023.

Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. MOReL: Model-based offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Yann LeCun. A path towards autonomous machine intelligence, 2022. OpenReview position paper.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

Robert G. Newcombe. Interval estimation for the difference between independent proportions: Comparison of eleven methods. *Statistics in Medicine*, 17(8):873–890, 1998.

Seohong Park et al. OGBench: Benchmarking offline goal-conditioned rl. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Anton M. Schäfer, Steffen Udluft, and Hans-Georg Zimmermann. The recurrent control neural network. *Engineering Applications of Artificial Intelligence*, 20(2):197–210, 2007.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind control suite, 2018.

Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqi Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa. dm_control: Software and tasks for continuous control, 2020.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. MOPO: Model-based offline policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.